Phase 1 Build Playbook

Build a real, working, pitchable Phase 1 version of the AI Work Verification Platform that can be shown to customers and investors.

This playbook covers scope, architecture, coding, UI, database, infrastructure, delivery responsibilities, Cursor usage, commands, and the main challenges with recommended solutions.

1. What Phase 1 Must Be

Phase 1 should be a working operational wedge, not the full company vision.

- connect PM and time systems
- import tasks, users, and time entries
- verify hours against activity evidence
- review approved vs flagged work
- generate invoice drafts from approved records

Phase 1 promise: Connect PM and time systems, verify hours against work evidence, and generate invoice drafts from approved records.

2. The Demo to Build

1. Admin signs in and creates a workspace
2. Admin connects Jira or ClickUp
3. Admin connects Harvest or Tempo
4. Admin connects QuickBooks or Xero
5. Data sync imports users, projects, tasks, and time entries
6. The verification engine labels entries as verified, needs review, or flagged
7. A manager reviews anomalies in the dashboard and queue
8. A finance user generates a draft invoice from approved work

3. Recommended Stack

Layer	Recommendation
Frontend	Next.js, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query
Backend API	NestJS with REST APIs
Database	PostgreSQL with Prisma
Jobs	Redis + BullMQ
Auth	Clerk or Auth0
Storage	S3 or MinIO for local/dev
Deployment	Vercel + Railway/Render/Fly + Neon + Upstash

Why this stack

It is fast to build, modern, manageable, and friendly for Cursor-assisted development.

Why not overbuild

Phase 1 needs speed, credibility, and a clean demo more than enterprise-scale infrastructure.

4. Architecture

- **Web app:** onboarding, integrations, dashboard, review queue, invoice draft UI
- **API service:** workspaces, integrations, sync orchestration, verification, invoice logic
- **Worker service:** background syncs, webhook processing, scoring, draft generation
- **Database:** multi-tenant relational store for normalized entities and auditability
- **Storage:** files, exports, and future document support

5. Repository Structure

```
phase1-platform/
  apps/
    web/
    api/
    worker/
  packages/
    ui/
    config/
    types/
    sdk/
  infra/
```

```
    docker/
    scripts/
  docs/
```

6. Core Screens

- sign-in and workspace setup
- integrations page
- dashboard
- verification review queue
- project detail page
- invoice draft page
- audit log/activity view

7. Data Model

Use strict workspace scoping and store both normalized data and source-provider references.

- workspaces, users, workspace_members
- integrations, external_identities
- projects, tasks, time_entries, activity_events
- verification_results, verification_reasons
- invoice_drafts, invoice_lines
- audit_logs

8. Core Backend Modules

- auth
- workspaces
- integrations
- sync
- users
- projects
- tasks
- time-entries
- verification
- invoices
- audit

9. Verification Engine

Use deterministic rules first. This makes Phase 1 explainable and trustworthy.

- linked task present → positive score
- task updated near the work window → positive score
- comments/activity present → positive score
- duplicate overlapping time entries → negative score
- task closed before time log → flag
- unmapped user → flag

Statuses: verified, needs_review, flagged.

10. Invoice Draft Engine

- input: approved time entries, client/project mapping, user or role rates
- output: draft invoice, line items, totals, summary note
- keep logic simple in Phase 1: hourly by user or role only

11. Team Workstreams

Track	Focus
Product and UX	golden flow, wireframes, status language, review UX, invoice layout
Platform and Infra	monorepo, environments, CI/CD, database, queues, secrets, deployments
Backend and Data	schema, APIs, verification, invoice engine, audit trail, tenant safety
Integrations	Jira/ClickUp, Harvest/Tempo, accounting connector shell, idempotent syncs
Frontend App	app shell, screens, tables, drawers, dashboard, invoice views
Demo and Pitch	demo data, screenshots, walkthrough, product story

12. Build Sequence

Bootstrap repository

```
mkdir phase1-platform
cd phase1-platform
pnpm init
pnpm dlx create-turbo@latest .
```

Create the web app

```
pnpm dlx create-next-app@latest apps/web --ts --tailwind --eslint --app --src-d
```

Create the API app

```
pnpm add -D @nestjs/cli
pnpm dlx @nestjs/cli new apps/api --package-manager pnpm
```

Create the worker and shared tooling

```
mkdir -p apps/worker/src packages/ui packages/types infra/docker
pnpm add -w @prisma/client zod dotenv bullmq ioredis
pnpm add -w -D prisma typescript tsx turbo eslint prettier
pnpm prisma init
```

Start local infrastructure

```
docker compose up -d postgres redis minio
pnpm prisma migrate dev --name init
pnpm prisma generate
```

Then build in this order

1. auth and workspace setup
2. integration framework
3. normalized entities
4. verification engine
5. invoice draft engine
6. frontend flows
7. demo seed data
8. staging deployment and golden-path demo

---

13. Main Challenges and Solutions

Challenge	Solution

Identity mapping across tools	Use external_identities, email-first matching, manual review/merge fallback.
API rate limits and sync failures	Use provider adapters, sync cursors, retries, and idempotent jobs.
Opaque verification scoring	Store reasons, show triggered rules, allow overrides, and log every review action.
Invoice logic exploding in complexity	Support only hourly user or role rates in Phase 1.
UI overload	Keep 5–7 core screens and use progressive disclosure.
Multi-tenant data leakage risk	Enforce workspace scope on every query and action.
Messy demos	Seed curated, realistic data and prepare a polished golden-path workspace.

14. How Cursor Should Be Used

Use Cursor as an execution accelerator for small, clear tasks.

- good for scaffolding modules, tables, DTOs, queue handlers, and tests
- good for schema drafts and refactors
- bad for inventing business logic, billing rules, or unsupported claims

15. Concrete Cursor Commands and Prompts

Bootstrap commands

```
pnpm install
docker compose up -d
pnpm prisma migrate dev
pnpm dev
```

Prompt: create Prisma schema

```
Create a Phase 1 Prisma schema for a multi-tenant work verification platform.
Include:
- workspaces
- users
- workspace_members
- integrations
- external_identities
```

```
- projects
- tasks
- time_entries
- activity_events
- verification_results
- invoice_drafts
- invoice_lines
- audit_logs
Requirements:
- every business table must be workspace-scoped
- include unique constraints for provider external ids
- optimize for Jira/Harvest style imports
```

Prompt: build verification engine

```
Implement the Phase 1 verification engine.
Rules:
- linked task present => positive score
- task updated near work window => positive score
- comments/activity present => positive score
- duplicate overlapping time entries => negative score
- task closed before time log => flag
- unmapped user => flag
Output:
- score
- status
- explanation reasons array
Add unit tests for the rules.
```

Prompt: build dashboard UI

```
Build the Phase 1 dashboard in Next.js.
Include:
- KPI cards for total hours, verified hours, flagged hours, invoiceable hours
- anomaly panel
- recent sync panel
- pending review table preview
Use Tailwind and reusable components.
Keep it visually clean and pitch-demo friendly.
```

Prompt: build invoice draft flow

```
Build the invoice draft feature for Phase 1.
Requirements:
- group approved time entries by project/client
- create line items using user or role hourly rate
- show total hours, rate, amount
- include a generated work summary field
- maintain references back to time entries
Implement API, service layer, and Next.js UI.
```

16. Definition of Done

- a demo user can onboard a workspace
- sample integrations sync realistic data
- the verification dashboard is clear
- flagged items can be reviewed and approved
- invoice drafts can be generated from approved work
- the app looks credible enough for customer and investor demos

17. Final Recommendation

Build Phase 1 as a credible working wedge: deterministic verification, simple invoice rules, curated demo data, managed infrastructure, and a small number of polished screens. That is the fastest path to something real, demoable, and fundable.